

Problem 1 Editorial

Duck

Subtask 1

Each color only has one valid painting company.

We can therefore divide the sequence N into segments s, e , which means that it is possible to cover all from s to e without changing the pattern.

The answer is simply $\sum \lceil \frac{e-s+1}{K} \rceil$

Subtask 2, 3

Let's model this problem as an interval covering problem. An interval $[y, y + M - 1]$ can be taken if there exists a valid rotation (i.e. choose a value of x) of the contractors such that each contractor is happy painting his section of the wall. For example, for the sample input 3 3 1 3 4 4 2 2, the valid intervals are $[0, 2]$, $[1, 3]$, $[2, 4]$ and $[5, 7]$.

We can find the list of valid intervals by brute force trying for each y , try all x . If there is at least one x in which all contractors are happy, then interval $[y, y + M - 1]$ is valid. This can be done in $O(NM^2)$.

Now that we have found the valid intervals, we just need to find the fewest intervals to cover the entire range. This can be solved various greedy algorithms.

Subtask 4

This subtask is similar to the final subtask, but not utilizing the 400000 condition.

There are multiple ways to solve this subtask. For example, we can explicitly find which worker can paint which wall in $O(NM)$. Note that if you are unfamiliar with the greedy method, you can use a slow dynamic programming method to solve this subtask.

Subtask 5

In this subtask, we need a more efficient way to check if an interval is valid. The idea here is to use a sliding window. When we consider moving from $[y, y + M - 1]$ to $[y + 1, y + M]$, most of the elements are still repeated. We only need to consider the effects of elements y and elements $y + M$.

For example, the following diagram illustrates transitioning from $[1, 3]$ to $[2, 4]$ in sample input 1. Color 1 is green, Color 3 is blue and Color 4 is red. Also the contractors 0, 1, 2 are now A, B, C respectively. We keep track for each rotation, the number of correct places (the number of contractors which are happy to paint that part of the wall.) When we transition from $[1, 3]$ to $[2, 4]$, notice that we only need to change the total based on position 1 and position 4.

Position	1	2	3	4
-----------------	----------	----------	----------	----------

A: Green **B:** Blue **C:** Blue & Red

A	B	C	total
1	2	3	1
3	1	2	1
2	3	1	3



A	B	C	total
4	2	3	1
3	4	2	0
2	3	4	3

Since $\sum f(k)^2 \leq 400000$, it means that each color only has at most $\sqrt{400000}$ occurrences. This means that each slide of the sliding window takes at most $O(\sqrt{400000})$. This gives the final complexity of $O(N\sqrt{400000})$

Problem 2 Editorial

Wabbit

Abridged Problem Statement

You are given weighted, connected graph with N vertices and M edges. Answer Q online queries of the form:

- Given 2 vertices X and Y , find the smallest positive integer C such that, using only edges with weight $\leq C$, there exists a way to have 2 cars starting at X and Y to travel to the other vertex without meeting at the same vertex or crossing the same edge in different directions.

Subtask 1

The graph is either a cycle or a line. If it is a line, then it is impossible no matter what the query is, since the "relative ordering" of the 2 cars will never change as they cannot cross each other directly.

If it is a cycle, then as a consequence of the line case, every edge in the cycle must be traversed. So the answer for any query on a cycle is the maximum of all edges in the cycle.

Subtask 6

Notice that if it is possible to get the cars across using a capacity of C , then it is also possible to do it with capacity $C + 1$. Similarly, if it not possible to do it with capacity C , it is not possible to do it with capacity $C - 1$. This means that we can binary search the optimal answer.

Now we fix the capacity at C , and we throw away all roads with weight more than C to get a new graph. We now prove the following claim.

Claim 1. *It is possible to get the cars at X and Y to each other if and only both X and Y both belong in the same connected component AND the connected component is not a line.*

Proof. Connectedness is clearly necessary. We know from Subtask 1 that if it is a line, there is no way for the cars to dodge each other.

If there is a cycle in this component, then it is always possible for both cars to travel 2 distinct vertices in the cycle. They can then "swap" positions by having one car go around the cycle, and now both cars can reach their destinations.

If the component is a tree, take any 2 leaves A and B whose path between them contains both X and Y and the path looks like $A - X - Y - B$ (possibly with vertices in between). Then there is a leaf Z that does not lie on the $A - B$ path. There are 2 cases, depending on where the path from Z connects to the $A - B$ path:

- (1) In between" X and Y . We can have one car hide in Z while the other car travels across
- (2) Behind X or Y . WLOG it is behind X . Have both cars travel to A and Z , then have the X car travel to Y and the Y car travel to X .

□

Now consider what happens when we increase C . This means adding some edges. Adding edges to any connected component that is not a line will never make it a line, and merging lines with non-lines will create non-lines. Additionally, merging lines with other lines only makes a line if the edge added consists of one endpoint from each line.

This motivates the following approach: Incrementally add edges to the graph in non-decreasing weight and maintain a union-find-disjoint-set (UFDS) data structure (with union by size to ensure $O(\log N)$ per operation) that record

- **par(x):**
Parent of the current node (initially itself)
- **mergeWeight(x):**
Edge weight when this node is no longer the representative of its set (the weight of the edge when its parent was changed from itself to another node)
- **size(x):**
Size of connected component that it represents
- **start(x), end(x):**
Endpoints of the line if the connected component is a line.
- **lineWeight(x):**
Edge weight when the connected component that this represents is no longer a line (-1 if it is still a line)

Maintaining these is fairly simple as the edges are merged. The only cases to note are (1) adding an edge between 2 nodes in the set, which instantly destroys any line as it creates a cycle and (2) checking whether the added edge connects the endpoints of 2 lines.

To answer each query, we can binary search the answer. When we try a possible capacity C , we traverse upwards along the tree formed by the UFDS from both X and Y until we reach a node where ‘mergeWeight(x)’ $> C$ which means we cannot move up anymore.

If the nodes are different, then we are not in the same connected component. Otherwise, we check if ‘lineWeight(x)’ $\leq C$. If it is, then C works. Otherwise, C doesn’t work.

In the worst case, this is $O(\log N \log 10^9)$ per query without any optimizations, since we do $O(\log 10^9)$ iterations for binary search and $O(\log N)$ for traversing the UFDS. This can be improved with discretization to $O(\log^2 N)$, but one can get AC with the naive implementation.

Problem 3 Editorial

Orange

Abridged Problem Statement

We are given a tree, with each node having a degree at most 3. We are given $4N$ queries, each of which are the following kind:

- Given nodes X and Y , query the distance between nodes X and Y .
- Given nodes A and B , query the size of the subtree of node B when rooted at node A .

We want to find a path through all the nodes, such that the distance between adjacent nodes is strictly increasing.

Subtasks 1 and 2

N is small, we can query the distance between all pairs of nodes, hence we query all pairs of nodes, and draw an edge iff the distance between 2 nodes is equal to 1. We can now repeatedly find the furthest away node from a current node. Since at every choice we remove the diameter of the tree, this is strictly non-increasing.

Intuition

If we want to greedily choose the furthest node, an issue may arise if 2 nodes are in the same subtree. Therefore, a way to prevent this would be to make sure any 2 consecutive nodes are from different subtrees. This would require no subtree to have more than $\frac{N}{2}$ nodes in its subtree, so we can think about dividing the tree based on its centroid.

Solution, Part 1

A centroid is defined as a node that if deleted, will split the tree into pieces size not more than half the size of a tree. The existence and proof are well-known. We can find the centroid by "rooting" the tree arbitrarily, and querying the size of the subtrees of all nodes.

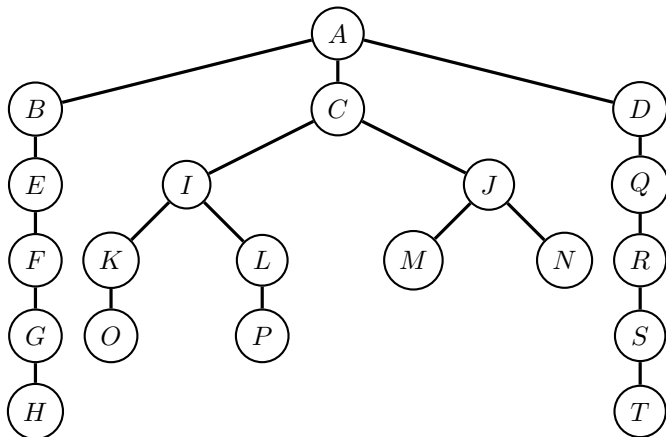
Normally, we would find a centroid by finding a node that has at least $\frac{N}{2}$ nodes in its subtree, but none of its children do. We claim that this would be the node with the smallest subtree size that is at least $\frac{N}{2}$.

Proof by contradiction: If this node is not a centroid, then it must split the tree into parts, at least one of which is sized $\frac{N}{2}$. Since there exists no other nodes with a smaller size that is bigger than $\frac{N}{2}$, and there are less than $\frac{N}{2}$ nodes not in its subtree, this node must be the centroid.

Hence we have found the centroid in $N - 1$ queries.

Solution, Part 2

We can take each of the nodes adjacent to the centroid, and find all the nodes in its subtree with a linear sweep of all nodes. There should be at most $2N$ queries needed to do this, as each node has at most 3 neighbours. We can then sort all the nodes by distance from the root, with the nodes with greater distances coming first. We can now greedily pick the subtree with the greatest height, add it to our answer and remove the node. However, this may fail at any point in time, a "dominant" subtree occurs with more than half of the remaining nodes, such as as follows:



This would select the nodes $H \rightarrow T \rightarrow G \rightarrow S$. This would leave the subtree under node C with more than half of the remaining nodes, causing the algorithm to fail. To counter this, before selecting each node, we check if the size of the biggest subtree is equal to the number of other nodes. If this happens, we can greedily "merge" all the other subtrees together. As such, by alternating between In this case, the path will be $H \rightarrow T(\text{Merge}) \rightarrow O \rightarrow G \rightarrow P \rightarrow S$. This will ensure that we do not have any subtree have majority of the remaining nodes.

Note that there will be an edge case, when the node furthest from the centroid is not in the current or biggest subtrees, and we may miss it out, leading to a WA. This can be rectified by jumping to the tallest subtree before we begin alternating.

Final complexity: $4N$ queries, $O(N \log N)$